Software Requirements Specification (SRS)

Remote Lab Reservation System (RLRS) — IEEE 830 / 29148 Standard

Project	Remote Lab Reservation System (RLRS)
Version	1.1
Authors	Team 11: Mahipal Singh (PES1UG24AM153, Team Leader), Deepesh M (PES1UG24AM150), Lakshya Budhauliya (PES1UG24AM147), Venkat Kushal (PES1UG24AM124)
Date	11 November 2026
Status	Updated to match the implemented system (v1.1)

Revision History

Version	Date	Author	Change summary	Approval
1.0	30 October 2026	Team 11	Initial project-specific IEEE baseline specification.	Approved
1.1	11 November 2026	Team 11	Aligned with implemented system: Python stdlib HTTP server + SQLite, built-in worker thread, custom week calendar, UTC timestamp persistence (+05:30 IST lab display), holder-only remote_info, embedded UML Use-Case diagram, and 71/71 passing automated test results in RTM.	Approved

Approvals

Role	Name	Signature / SRN / Email	Date
Project team representative	Mahipal Singh	PES1UG24AM153 (Team Leader)	11 November 2026
Scheduling & backend lead	Lakshya Budhauliya	PES1UG24AM147	11 November 2026
Calendar UI, auth & email lead	Deepesh M	PES1UG24AM150 (mdeepesh2006@gmail.com)	11 November 2026
QA & documentation member	Venkat Kushal	PES1UG24AM124	11 November 2026
Course instructor / TA	Kshitij Shetty	SE Teaching Assistant, Dept. of CSE(AIML)	Pending Review

Table of Contents

- 1. Introduction (1.1 Purpose, 1.2 Scope, 1.3 Audience, 1.4 Definitions)
- 2. Overall Description (2.1 Product Perspective, 2.2 Major Product Functions, 2.3 User Roles, 2.4 Operating Environment, 2.5 Constraints & Assumptions)
- 3. External Interface Requirements (3.1 User Interface, 3.2 Software Interfaces, 3.3 Communications)
- 4. System Features / Functional Requirements (RLRS-F-001 to RLRS-F-022)
- 5. Non-Functional Requirements (RLRS-NF-001 to RLRS-NF-008)
- 6. Security Objectives and Requirements (6.1 Security Objectives, 6.2 Security Requirements RLRS-SR-001 to RLRS-SR-007)
- 7. Quality Attributes and Acceptance Tests
- 8. UML Use-Case Models (Visual UML Use-Case Diagram & Specification)
- 9. Requirements Traceability Matrix (RTM — Verified against 71 Automated Tests)
- 10. Deviations from v1.0 Baseline

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, and security requirements for the Remote Lab Reservation System (RLRS), a web-based booking portal through which students reserve timeslots on lab equipment, view calendar availability, and receive automated email reminders. It serves as the authoritative reference for implementation, testing, and evaluation.

1.2 Scope

RLRS supports managing labs and equipment, configuring bookable hours and slot lengths, showing availability on a calendar, creating, approving and cancelling reservations, detecting and preventing scheduling conflicts, blocking slots for maintenance, sending confirmation and reminder emails, recording attendance, and reporting on equipment utilisation. The submitted v1.1 build implements the scope above with the documented deviations in Section 10. Faculty and Viewer roles, separate day calendar view, iCal export, SMS, SSO, and production HTTPS deployment are outside the submitted build.

1.3 Audience

Student developers (Team 11), testers, users acting as students, lab in-charges and administrators, project evaluators, and the teaching assistant.

1.4 Definitions

Term	Definition
RLRS	Remote Lab Reservation System
Equipment	A single bookable lab resource (instrument, workstation, kit or remote-access setup) belonging to one lab
Slot	A bookable time interval on the lab's calendar grid, defined by the lab's operating hours and slot length
Reservation	A booking of one equipment item by one user for a start and end time, with a lifecycle status
Conflict	Two active reservations, or a reservation and a maintenance block, on the same equipment whose half-open time ranges [start, end) overlap
Maintenance block	A time range during which equipment is unavailable for booking; automatically cancels overlapping active reservations

Term	Definition
Booking rules	Configurable per-lab limits: operating hours, slot length, advance-booking window, maximum duration, maximum active bookings per student, and 60-minute cancellation cut-off
No-show (NoShow)	A confirmed reservation that the user did not attend, marked by lab staff after the slot ends
REST / RBAC	Representational State Transfer JSON HTTP API / Role-Based Access Control

2. Overall Description

2.1 Product Perspective

RLRS is a self-contained web application comprising a browser-based weekly calendar user interface, a RESTful JSON API server, a pure-Python scheduling engine (conflict detection and booking rules), a background worker thread for email and reminder dispatch, and a persistent SQLite database.

2.2 Major Product Functions

- Authenticate users with salted PBKDF2-SHA256 password hashes and enforce server-side role-based permissions.
- Create and maintain labs, equipment, operating hours, slot lengths, and per-lab booking limits.
- Display equipment availability in an interactive weekly calendar grid with text and colour indicators.
- Create reservations on free slots inside an atomic database transaction with conflict detection and booking-rule validation.
- Route reservations for approval-required equipment to that lab's assigned Lab In-charge for approval or rejection.
- Allow users to view their reservations, see remote-access instructions for confirmed bookings, and cancel before the 60-minute cut-off.
- Block equipment for maintenance and automatically cancel and email affected reservation holders.
- Send confirmation, cancellation, approval/rejection, and automated 2-hour advance reminder emails with retry/backoff.
- Record attendance (Completed or NoShow) after each slot and generate equipment utilisation and audit reports.

2.3 User Roles and Characteristics

Role	Needs / permissions (as implemented in v1.1)
Administrator (Admin)	Manage all labs, equipment, maintenance blocks, approvals and attendance; view all reservations, utilisation report and audit log.
Lab in-charge (LabInCharge)	Manage equipment, operating hours, slot length and booking limits for labs they are assigned to; approve or reject restricted bookings; create maintenance blocks; mark attendance; view reports and audit logs.
Student	Self-register; view availability calendar; create, view and cancel own reservations; view remote-access details for own confirmed reservations; receive confirmation and reminder emails.

Note: Faculty and Viewer roles from the v1.0 draft were consolidated; faculty can use a Student account and evaluators can use the seeded demo accounts.

2.4 Operating Environment

RLRS runs in any modern desktop browser (verified with Chromium via Playwright). The backend server is a single Python program (verified on Python 3.12, 3.13, and 3.14 across Ubuntu Linux, Windows, and macOS) using only the Python standard library (http.server, sqlite3, smtplib, threading, hashlib), requiring zero third-party runtime packages. Data is stored in a local SQLite database file (rlrs.db). A background worker thread inside the

server process dispatches confirmation, cancellation, and reminder emails via SMTP (when SMTP_HOST is configured) or writes them to the local outbox/ directory during development.

2.5 Constraints and Assumptions

- Each reservation is for one equipment item; booking multiple items requires separate reservations.
- Reservations must start and end on the lab's configured slot boundaries.
- Time ranges are half-open [start, end): back-to-back reservations (one ending at 10:00 and the next starting at 10:00) do not conflict.
- Timestamps are stored in SQLite in UTC. The API, UI, and emails use the lab's local time (fixed offset +05:30 IST by default, configurable via RLRS_UTC_OFFSET).
- Technologies were adapted from the v1.0 proposal so the application runs identically across Windows, macOS, and Linux without external database servers; all deviations are recorded in Section 10.

3. External Interface Requirements

3.1 User Interface

- Interactive week-view calendar grid (Monday–Sunday with Prev week / This week / Next week navigation) per equipment item, showing Free, Yours, Booked, Maintenance, and Past slots using both text labels and colour.
- Selecting a free slot opens a pre-filled booking form with duration selector.
- 'My reservations' tab listing upcoming and past bookings, status badges, remote-access instructions (for Confirmed bookings), and Cancel button.
- Staff 'Manage' and 'Reports' tabs for pending approvals, attendance marking, equipment CRUD, lab hours/rules configuration, maintenance blocks, utilisation reports, and audit logs.

3.2 Software Interfaces & 3.3 Communications

- RESTful JSON HTTP API (/api/health, /api/login, /api/register, /api/labs, /api/equipment, /api/reservations, /api/maintenance, /api/reports/utilisation, /api/audit).
- SQLite relational database storing users, sessions, labs, equipment, reservations, maintenance, reminder_jobs, email_jobs, and audit tables.
- Email delivery through SMTP (SmtpTransport) when configured or local outbox/ files (FileTransport) for deterministic development and demo verification.

4. System Features / Functional Requirements

Req ID	Requirement (shall)	Priority	Acceptance criteria
RLRS-F-001	The system shall allow an authorized user to sign in and sign out.	High	Valid credentials create an HttpOnly session; invalid credentials return 401.
RLRS-F-002	The system shall enforce role-based permissions for student, lab in-charge and administrator actions.	High	Unauthorized role receives 403 Forbidden on direct API requests.
RLRS-F-003	The system shall allow authorized staff to create, edit and deactivate equipment (name, lab, type, description, active, approval required, remote info).	High	Valid equipment is saved and searchable; deactivated equipment is hidden from students and cannot be booked (409).
RLRS-F-004	The system shall allow a lab in-charge or admin to configure each lab's operating hours and slot length.	High	Calendar generates bookable slots strictly within operating hours at the configured slot length.

Req ID	Requirement (shall)	Priority	Acceptance criteria
RLRS-F-005	The system shall allow users to browse and filter equipment by lab, type and search query.	Medium	GET /api/equipment?lab_id=&type;=&q;= returns matching active equipment.
RLRS-F-006	The system shall display equipment availability in a week calendar view.	High	Each slot is rendered as free, booked, own, maintenance, or past with text label and colour.
RLRS-F-007	The system shall allow a signed-in user to create a reservation for one equipment item on slot boundaries.	High	Valid request returns 201 with Confirmed (or Pending if approval required).
RLRS-F-008	The scheduling engine shall reject a reservation that overlaps an active reservation or maintenance block on the same equipment.	High	Overlapping requests return 409 CONFLICT with conflicting time range; back-to-back bookings succeed.
RLRS-F-009	The scheduling engine shall enforce booking rules: future start, inside operating hours, on slot grid, within advance window, within max duration, within max active bookings, and no same-user overlap.	High	Rule violations return 400 RULE_VIOLATION naming every violated rule code.
RLRS-F-010	The system shall place approval-required bookings in Pending status and allow that lab's in-charge to approve or reject (with required reason).	Medium	Approve transitions to Confirmed and schedules reminder; reject requires reason, frees slot, and emails holder.
RLRS-F-011	The system shall show each user their upcoming and past reservations with current status.	High	GET /api/reservations/mine matches stored reservations for the signed-in user.
RLRS-F-012	The system shall allow a user to cancel their own reservation at least 60 minutes before start (staff may cancel anytime).	High	Cancellation frees slot and cancels pending reminder; student cancel within 60 min returns 409 CUTOFF_PASSED.
RLRS-F-013	The system shall allow lab staff to create maintenance blocks and shall automatically cancel and notify holders of overlapping reservations.	Medium	Overlapping Pending/Confirmed bookings become Cancelled, reminders are cancelled, and holders are emailed.
RLRS-F-014	The system shall queue an email when a reservation is confirmed, approved, rejected, or cancelled.	High	One email job per lifecycle event is persisted and dispatched to the holder.
RLRS-F-015	The system shall send an automated reminder email 2 hours before each confirmed reservation starts.	High	Reminder is sent exactly once when due, only if reservation remains Confirmed.
RLRS-F-016	The system shall retry failed email/reminder sends up to 3 attempts with backoff and record final status.	Medium	Transient failure increments attempts and retries; 3 failures mark status Failed.
RLRS-F-017	The system shall show equipment remote-access instructions only to the holder of a Confirmed reservation (and lab staff).	Medium	Confirmed holder sees remote_info; other students and Pending/Cancelled bookings see empty string.
RLRS-F-018	The system shall allow lab staff to mark a finished Confirmed reservation as Completed or NoShow.	Medium	Marking before slot end returns 409; valid outcome updates status and writes audit entry.
RLRS-F-019	The system shall generate utilisation reports (booked hours, available hours, %, bookings, cancellations, no-shows) for a date range.	Medium	Report numbers reconcile with stored reservations and lab hours.
RLRS-F-020	The system shall allow lab staff to configure per-lab booking rules (hours, slot length, max duration, advance days, max active).	Medium	Updated settings persist and govern subsequent reservations.

Req ID	Requirement (shall)	Priority	Acceptance criteria
RLRS-F-021	The system shall record an append-only audit event for logins, reservations, approvals, maintenance, attendance, and settings changes.	High	Audit entries record actor, action, UTC timestamp, and redacted details.
RLRS-F-022	The system shall return structured JSON validation and operation errors without exposing stack traces.	High	Errors return {error, message, details} with appropriate 4xx/5xx status and no Python traceback.

5. Non-Functional Requirements

Req ID	Requirement	Category	Acceptance / Measurement
RLRS-NF-001	For a dataset of 200 equipment items and 20,000 reservations, a week calendar view and booking request should complete within 2 seconds for 90% of requests.	Performance	Target specified; local response time on demo dataset is <50 ms (bulk 20k dataset not run).
RLRS-NF-002	When multiple users request the same equipment for overlapping times simultaneously, exactly one request shall succeed.	Integrity / Concurrency	Verified by 10-thread barrier concurrency test (1 succeeds, 9 receive 409 CONFLICT).
RLRS-NF-003	The application shall preserve committed reservations across restarts and resume pending reminders without duplicate sends.	Reliability	Verified by SQLite persistence and idempotent reminder_jobs status test.
RLRS-NF-004	When the email service is available, due reminder emails shall be handed to it within 5 minutes of scheduled time.	Timeliness	Worker thread polls every 30 seconds; verified with controllable clock tests.
RLRS-NF-005	Passwords shall be stored as salted hashes (never plaintext) and all protected endpoints shall require authentication and role authorization.	Security	Verified by PBKDF2-SHA256 inspection, HttpOnly cookie check, and 401/403 API tests.
RLRS-NF-006	The calendar and forms shall provide clear navigation, readable labels, validation feedback, and slot states that do not rely on colour alone.	Usability / Accessibility	Verified by 9-step Playwright Chromium UI test; slots display text labels (Free, Yours, Booked, Maintenance).
RLRS-NF-007	The codebase shall isolate pure scheduling logic (rlrs_core) from HTTP routes and SQLite persistence so it can be unit-tested independently.	Maintainability	Verified by 37 fast unit tests in tests/unit/ with zero database or network I/O.
RLRS-NF-008	The application shall provide a documented setup procedure and run across Windows, macOS, and Linux.	Portability	Verified by GitHub Actions CI matrix (ubuntu-latest, windows-latest, macos-latest) and local macOS run.

6. Security Objectives and Requirements

6.1 Security Objectives

- Objective 1 (Confidentiality & Access Control): Protect user accounts, session tokens, reservation records, and equipment remote-access credentials from unauthorized viewing, spoofing, or modification.
- Objective 2 (Integrity & Non-Repudiation): Preserve the transactional integrity and audit traceability of all bookings, cancellations, approvals, and maintenance blocks so slot disputes can be investigated.
- Objective 3 (Input Safety & Secret Hygiene): Prevent injection, path traversal, stack-trace leakage, and credential exposure in source code, database tables, or audit logs.

6.2 Security Requirements

Req ID	Requirement	Priority	Verification / Test
RLRS-SR-001	The system shall require authentication before allowing any protected API route or reservation action.	High	test_requires_login_and_session_lifecycle (Pass)
RLRS-SR-002	The system shall enforce role-based authorization in the backend service layer, not merely by hiding UI tabs.	High	test_role_enforcement_is_server_side, test_student_cannot_approve (Pass)
RLRS-SR-003	The system shall store passwords using PBKDF2-HMAC-SHA256 (120,000 iterations, random 16-byte salt) and never log passwords.	High	test_register_login_and_bad_credentials, test_audit_never_contains_password (Pass)
RLRS-SR-004	The system shall validate and parameterize all user inputs (IDs, ISO dates/times, JSON bodies, static file paths) on the server.	High	test_sql_injection_strings_are_inert, test_static_page_served_and_path_traversal_blocked (Pass)
RLRS-SR-005	The system shall enforce record-level ownership: students may cancel only their own bookings, only confirmed holders see remote_info, and lab in-charges manage only their assigned labs.	High	test_student_cannot_cancel_someone_elses_reservation, test_rlrs8_remote_info_restricted_to_confirmed_holder_and_staff, test_rlrs11_other_lab_incharge_denied (Pass)
RLRS-SR-006	SMTP credentials and configuration secrets shall be read exclusively from environment variables and never committed to Git.	High	Repository inspection + mailer.py environment configuration (Pass)
RLRS-SR-007	The system shall record audit events for privileged and reservation actions while automatically redacting password/token/secret keys.	High	test_audit_records_actor_action_time_and_redacts_secrets (Pass)

7. Quality Attributes and Acceptance Tests

Quality Attribute	Acceptance Criterion & Verification Result
Correctness	All conflict boundary cases (overlap, containment, back-to-back) and all 7 booking rules verified across 37 unit tests and 34 integration/API tests (71/71 Passed).
Integrity	10 simultaneous threads booking the same equipment slot inside BEGIN IMMEDIATE transaction result in exactly 1 Confirmed booking and 9 CONFLICT responses.
Reliability	Pending reminder and email jobs persist in SQLite across restarts, retry up to 3 times on SMTP failure, and send at most once.
Security	PBKDF2 password hashing, HttpOnly + SameSite=Lax cookies, server-side RBAC, cross-lab isolation, holder-only remote_info, SQLi and path traversal defences verified.
Portability	GitHub Actions CI passes all 71 tests automatically on Ubuntu Linux, Windows, and macOS.

8. UML Use-Case Models

Figure 8.1 illustrates the UML Use-Case Diagram for RLRS, showing the three human actors (Student, Lab In-charge inheriting Student capabilities, and Administrator inheriting Lab In-charge capabilities) and the secondary Email Service (SMTP) actor, along with <<include>> relationships for conflict detection and booking-rule validation and <<extend>> for maintenance auto-cancellation.

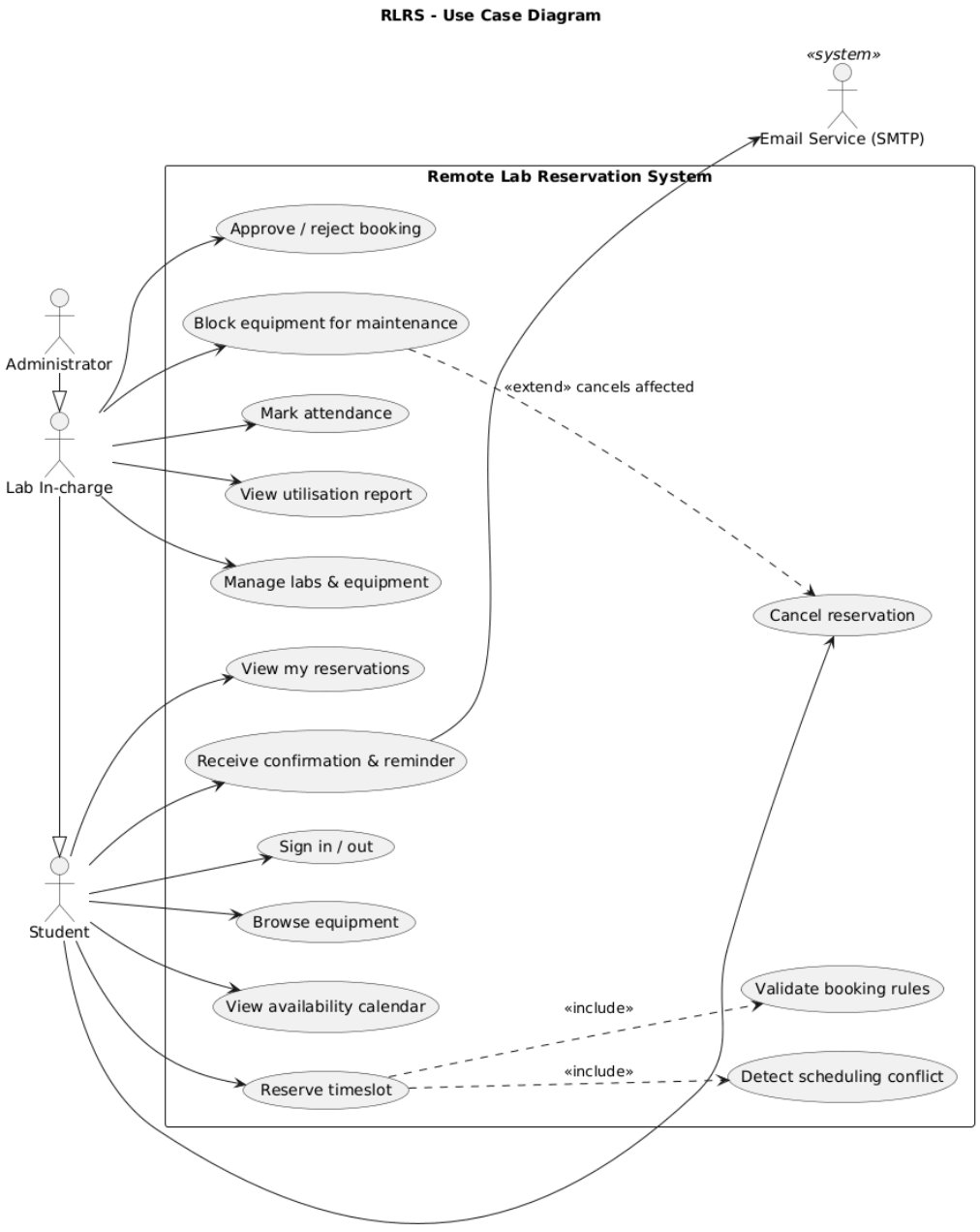


Figure 8.1: RLRS UML Use-Case Diagram (01_use_case.puml)

9. Requirements Traceability Matrix (RTM)

Req ID	Short Requirement	Module (as built)	Automated Test(s)	Status
RLRS-F-001	Sign in/out	service.login, server.py	test_register_login_and_bad_credentials, test_requires_login_and_session_lifecycle	Passed (automated)
RLRS-F-002	Role-based access	service.py role checks	test_role_enforcement_is_server_side, test_student_cannot_approve	Passed (automated)
RLRS-F-003	Manage equipment	service.create/update_equipment	test_equipment_management_permissions_and_search, test_inactive_equipment_cannot_be_booked	Passed (automated)
RLRS-F-004	Hours & slot length	service.update_lab	test_lab_hours_change_drives_the_calendar_grid	Passed (automated)
RLRS-F-005	Browse/search equipment	GET /api/equipment	test_equipment_management_permissions_and_search	Passed (API); UI uses dropdown
RLRS-F-006	Availability calendar	service.availability, app.js	test_availability_marks_own_booked_and_validates_range, UI steps 2 & 6	Passed (week view)
RLRS-F-007	Create reservation	service.create_reservation	test_booking_creates_reservation_email_reminder_audit, test_booking_over_http_201_400_409	Passed (automated)
RLRS-F-008	Conflict detection	rlrs_core/conflict.py	9 unit tests in test_conflict.py, test_conflict_returns_409_with_range_and_rolls_back	Passed (automated)
RLRS-F-009	Booking rules	rlrs_core/rules.py	10 unit tests in test_rules.py, test_rule_violations_return_400_with_codes	Passed (automated)
RLRS-F-010	Approval workflow	service.approve/reject_reservation	test_approval_flow_and_rejection, test_approval_flow_over_http	Passed (automated)
RLRS-F-011	My reservations	GET /api/reservations/mine	test_booking_over_http_201_400_409, UI step 4	Passed (automated & UI)
RLRS-F-012	Cancel reservation	service.cancel_reservation	test_cancel_frees_slot_and_cancels_reminder, test_cancel_after_cutoff_refused	Passed (automated)
RLRS-F-013	Maintenance block	rlrs_core/maintenance.py, service.add_maintenance	test_maintenance_cancels_overlapping_and_blocks_new_bookings, UI step 5	Passed (automated)
RLRS-F-014	Lifecycle event emails	service._email, mailer.py	test_booking_creates_reservation_email_reminder_audit, test_emails_flushed_once_and_retried_on_failure	Passed (FileTransport)
RLRS-F-015	Reminder email (2h)	rlrs_core/reminders.py	test_reminder_sent_two_hours_before_exactly_once_and_survives_restart, 9 unit tests	Passed (automated)
RLRS-F-016	Retry failed emails	service.flush_emails, reminders.py	test_temporary_failure_is_retried_then_succeeds, test_repeated_failure_marks_failed	Passed (automated)
RLRS-F-017	Remote-access holder only	service.list_equipment, _res_dict	test_rlrs8_remote_info_restricted_to_confirmed_holder_and_staff	Passed (automated)
RLRS-F-018	Attendance marking	rlrs_core/attendance.py	test_attendance_only_after_slot_end_and_audited, 3 unit tests	Passed (automated)

Req ID	Short Requirement	Module (as built)	Automated Test(s)	Status
RLRS-F-019	Utilisation report	rlrs_core/report.py	test_utilisation_report_reconciles, 3 unit tests, UI step 7	Passed (automated)
RLRS-F-020	Configure lab rules	PUT /api/labs/{id}	test_lab_hours_change_drives_the_calendar_grid	Passed (per-lab rules)
RLRS-F-021	Audit log	rlrs_core/audit.py	test_audit_records_actor_action_time_and_redacts_secrets, test_audit_never_contains_password	Passed (automated)
RLRS-F-022	Safe JSON errors	server.py error handler	test_errors_are_json_without_stack_traces	Passed (automated)
RLRS-NF-001	Performance (20k rows)	SQLite indexed queries	none (bulk dataset not generated)	Not tested
RLRS-NF-002	No double booking	BEGIN IMMEDIATE tx	test_concurrent_requests_for_same_slot_yield_exactly_one_booking	Passed (automated)
RLRS-NF-003	Survive restart, once	SQLite reminder_jobs	test_reminder_sent_two_hours_before_exactly_once_and_survives_restart	Passed (automated)
RLRS-NF-004	Reminder within 5 min	worker_loop every 30s	test_reminder_sent_two_hours_before_exactly_once_and_survives_restart	Passed (automated)
RLRS-NF-005	Secure credentials	security.py PBKDF2	test_audit_never_contains_password, test_cookie_is_httponly	Passed (automated)
RLRS-NF-006	Usability & labels	rlrs_app/static/app.js	Playwright UI smoke steps 1-9	Passed (browser test)
RLRS-NF-007	Engine testable alone	rlrs_core/ (no I/O)	37 unit tests in tests/unit/	Passed (automated)
RLRS-NF-008	Documented setup	README.md, rlrs-ci.yml	CI on Ubuntu, Windows, macOS + local macOS run	Passed (CI + macOS)
RLRS-SR-001..007	AuthN/AuthZ, hashing, SQLi, path traversal, ownership, env secrets, audit redaction	security.py, service.py, server.py, audit.py	test_role_enforcement_is_server_side, test_sql_injection_strings_are_inert, test_static_page_served_and_path_traversal_blocked, test_rlrs8_remote_info_restricted_to_confirmed_holder_and_staff, test_rlrs11_other_lab_incharge_denied	Passed (automated)

10. Deviations from v1.0 Baseline

Area	v1.0 Plan	v1.1 As Built	Engineering Rationale
Web framework	FastAPI / Flask	Python stdlib http.server (ThreadingHTTPServer)	Zero external dependencies; runs out-of-the-box on Windows, macOS, and Linux.
Database	PostgreSQL (SQLite dev)	SQLite with BEGIN IMMEDIATE write locks	Portable single-file DB; BEGIN IMMEDIATE serializes check-then-insert (verified with 10-thread concurrency test).
Job scheduler	APScheduler / Celery	Background daemon thread in server.py (30s tick)	Single-process deployment with persistent SQLite job queues.
Calendar UI	FullCalendar (day + week)	Custom vanilla HTML/CSS/JS week grid	Dependency-free frontend; week view covers all booking and slot-state workflows.

Area	v1.0 Plan	v1.1 As Built	Engineering Rationale
Roles	Admin, LabInCharge, Faculty, Student, Viewer	Admin, LabInCharge, Student	Scope streamlined; Faculty bookings use Student role and evaluators use demo accounts.
Time storage	UTC with zoneinfo	UTC in SQLite; fixed offset (+05:30 IST default via RLRS_UTC_OFFSET) for display	Avoids tzdata package requirement on Windows while storing all DB timestamps in UTC.